

Практичне завдання №3

Дослідження ефективності TF-IDF зважених ембеддингів для детекції текстів, згенерованих великими мовними моделями

Мета роботи

Засвоїти принципи побудови векторних представлень текстів на основі субсловних ембеддингів (FastText), дослідити вплив TF-IDF зважування на якість агрегації документних векторів, порівняти ефективність лінійних та нелінійних методів класифікації (SVM, Logistic Regression, KNN) у задачі бінарної детекції штучно згенерованих текстів, а також проаналізувати геометричні властивості отриманих векторних просторів.

1 Теоретичні відомості

1.1 Ембеддинги (FastText)

На відміну від класичного Word2Vec, який будує векторні представлення для цілих слів, алгоритм FastText розглядає кожне слово як множину символьних n -грам. Формально, для слова w довжиною L та фіксованого n , множина n -грам визначається як:

$$\mathcal{G}(w) = \{g_i \mid g_i = w[i : i + n], i = 1, \dots, L - n + 1\} \cup \{w\} \quad (1)$$

Вектор слова обчислюється як сума векторів його n -грам:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}(w)} \mathbf{z}_g \quad (2)$$

Такий підхід забезпечує стійкість до слів, що не зустрічалися у навчальному корпусі (Out-Of-Vocabulary), оскільки навіть для невідомих слів можна побудувати множину n -грам.

1.2 TF-IDF зважування

Статистика TF-IDF (Term Frequency – Inverse Document Frequency) для терміна t у документі d визначається як:

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D) \quad (3)$$

де $\text{tf}(t, d)$ — частота терміна в документі, а

$$\text{idf}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (4)$$

де $|D|$ — загальна кількість документів у корпусі.

1.3 Агрегація документних векторів

Нехай документ d складається з токенів $w_1, w_2, \dots, w_m$. Документний вектор $\mathbf{v}_d$ може бути отриманий двома способами:

Mean Pooling (базовий):

$$\mathbf{v}_d^{\text{mean}} = \frac{1}{m} \sum_{i=1}^m \mathbf{v}_{w_i} \quad (5)$$

TF-IDF Weighted Mean (досліджуваний):

$$\mathbf{v}_d^{\text{tfidf}} = \frac{\sum_{i=1}^m \text{tfidf}(w_i, d, D) \cdot \mathbf{v}_{w_i}}{\sum_{i=1}^m \text{tfidf}(w_i, d, D)} \quad (6)$$

1.4 Методи класифікації

Support Vector Machine (SVM) з ядром $K(\mathbf{x}_i, \mathbf{x}_j)$ шукає гіперплощину, що максимізує зазор між класами. Функціонал оптимізації:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad (7)$$

Logistic Regression моделює ймовірність класу через сигмоїду:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x} + b))} \quad (8)$$

K-Nearest Neighbors (KNN) відносить об'єкт до класу більшості серед k найближчих сусідів у метричному просторі.

2 Опис датасету

Для виконання роботи використовується датасет *AI vs Human Text Classification Dataset 2026* (Kaggle), який містить тексти двох класів:

- **Human** — тексти, написані людьми;
- **AI** — тексти, згенеровані великими мовними моделями.

3 Методологія дослідження

3.1 Дослідницька гіпотеза

Великі мовні моделі, генеруючи тексти, зберігають семантичну відповідність тематиці (що відображається у множині використаних слів), проте залишають статистичні “відбитки” у використанні специфічної лексики — канцеляризмів, складних сполучників, характерних вставних конструкцій.

Гіпотеза: TF-IDF зважування підсилює вплив рідкісних, специфічних для III термінів, що призводить до кращої лінійної роздільності класів у порівнянні з простим Mean Pooling, де домінують часті загальноновживані слова, спільні для обох класів.

3.2 Експериментальний план

Дослідження передбачає факторний аналіз із двома незалежними змінними:

1. **Метод агрегації:** Mean Pooling vs. TF-IDF Weighted Mean;
2. **Класифікатор:** SVM (RBF kernel), Logistic Regression, KNN.

Залежні змінні: F1-score (macro), Accuracy, Balanced Accuracy, ROC-AUC.

4 Етапи виконання

4.1 Етап 1. Завантаження та первинний аналіз даних

1. Завантажити датасет та проаналізувати його структуру.
2. Перевірити баланс класів, розподіл довжин текстів.
3. Виявити можливі артефакти (дублікати, порожні рядки, надмірно короткі тексти).
4. Побудувати гістограми розподілу кількості слів для кожного класу.

4.2 Етап 2. Попередня обробка тексту (Spacy)

Реалізувати пайплайн обробки з використанням бібліотеки spacy:

```
1  import spacy
2
3  nlp = spacy.load("en_core_web_sm")
4
5  def preprocess(text: str) -> list:
6      doc = nlp(text)
7      tokens = []
8      for token in doc:
9          if token.is_alpha and not token.is_stop:
10             tokens.append(token.lemma_.lower())
11     return tokens
12
```

Вимоги до пайплайну:

- Лематизація (зведення до початкової форми);
- Видалення стоп-слів;
- Видалення небуквених символів;
- Зведення до нижнього регістру.

Примітка: Для пришвидшення обробки рекомендується використовувати `nlp.pipe()` з `batching`.

4.3 Етап 3. Навчання FastText моделі

З використанням бібліотеки `gensim` навчити модель FastText на обробленому корпусі:

```

1  from gensim.models import FastText
2
3  model = FastText(
4      sentences=corpus,
5      vector_size=100,
6      window=5,
7      min_count=2,
8      sg=1,
9      workers=4,
10     seed=42
11 )
12

```

4.4 Етап 4. Векторизація документів

Реалізувати два методи агрегації:

```

1  import numpy as np
2  from sklearn.feature_extraction.text import TfidfVectorizer
3
4  def mean_pooling(tokens, model):
5      vectors = [model.wv[t] for t in tokens if t in model.wv]
6      if not vectors:
7          return np.zeros(model.vector_size)
8      return np.mean(vectors, axis=0)
9
10 def tfidf_weighted_pooling(tokens, model, tfidf_scores):
11     vectors, weights = [], []
12     for t in tokens:
13         if t in model.wv and t in tfidf_scores:
14             vectors.append(model.wv[t])
15             weights.append(tfidf_scores[t])
16     if not vectors:
17         return np.zeros(model.vector_size)
18     weights = np.array(weights)
19     weights /= weights.sum()
20     return np.average(vectors, axis=0, weights=weights)
21

```

4.5 Етап 5. Навчання та оцінка класифікаторів

Для кожного методу агрегації навчити три класифікатори:

```
1 from sklearn.svm import SVC
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import (f1_score, accuracy_score,
6 balanced_accuracy_score, roc_auc_score)
7
8 X_train, X_test, y_train, y_test = train_test_split(
9 X, y, test_size=0.2, random_state=42, stratify=y
10 )
11
12 classifiers = {
13     'SVM': SVC(kernel='rbf', probability=True, random_state=42),
14     'LogReg': LogisticRegression(max_iter=1000, random_state=42),
15     'KNN': KNeighborsClassifier(n_neighbors=5)
16 }
17
```

4.6 Етап 6. Візуалізація та аналіз

1. Застосувати t-SNE або UMAP для проекції документних векторів у 2D-простір.
2. Побудувати окремі графіки для Mean Pooling та TF-IDF Weighted Mean.
3. Візуалізувати матриці плутанини (confusion matrices) для кожного класифікатора.

5 Дослідницькі питання

У звіті необхідно надати обґрунтовані відповіді на такі питання:

1. **Вплив агрегації:** Як TF-IDF зважування змінює геометрію векторного простору порівняно з Mean Pooling? Чому це призводить (або не призводить) до покращення лінійної роздільності?
2. **Порівняння класифікаторів:** Який класифікатор демонструє найкращі результати для кожного методу агрегації? Як це співвідноситься з природою даних (лінійна/нелінійна роздільність)?
3. **Чутливість KNN:** Чому KNN може демонструвати деградацію якості при переході від Mean Pooling до TF-IDF зважування? Як це пов'язано з “прокляттям розмірності” та зміною метричних властивостей простору?
4. **Лінгвістична інтерпретація:** Які n -грами або слова мають найбільші TF-IDF значення для класу AI? Чи можна їх інтерпретувати як “маркери” штучного тексту?
5. **Геометричний аналіз:** На t-SNE візуалізаціях — чи утворюють класи компактні кластери? Який метод агрегації забезпечує кращу кластеризацію?